

CSCE 627 THEORY OF COMPUTABILITY · MODULE 05 · LECTURE NOTES

Undecidability

The halting problem, proved.

MODULE OUTCOMES

Prove the halting problem undecidable by diagonalisation.

Explain the self-reference and why it is not a trick.

Prove that the complement of halting is not recognisable.

Explain the diagonal method in general.

State precisely what the result forbids.

1 The proof

```
HALT = { (M, w) : machine M halts on input w }
```

Suppose H decides HALT. Then build D:

```
D(X):                                # X is a machine encoding
  if H(X, X) says "halts":
    loop forever
  else:
    halt
```

D takes a machine description and does the OPPOSITE of what that machine does when run on its own description.

NOW RUN D ON ITS OWN DESCRIPTION, D(D):

```
if D halts on D, then H(D,D) says "halts",
  so by D's code D loops forever.    CONTRADICTION.
if D does not halt on D, then H(D,D) says "loops",
  so by D's code D halts.             CONTRADICTION.
```

Both branches contradict. So D cannot exist. But D was built from H by an entirely mechanical construction -- one call, one if-statement, and one loop. So H cannot exist.

THEREFORE HALT IS UNDECIDABLE.

That is the whole proof: four lines of construction and two cases.

Note carefully where the impossibility lands. D is unobjectionable code — a function call, a conditional, and a loop — **so the contradiction cannot be blamed on D. It falls on H**, the only assumed object in the argument. **That is the step to be certain of**, and it is where most first readings go wrong: the proof is not showing that self-reference is paradoxical, it is showing that *if H existed then a contradictory program would exist*, and therefore H does not.

THE SELF-REFERENCE IS NOT A TRICK

D is given its own description as a string. That is not a paradox and not sleight of hand — **it is Module 04 §3's encoding, used exactly as intended.**

You do this routinely. A compiler compiling its own source code; a diff of a file against itself; a hash of an executable computed by a program in that executable; a program printing its own source. **Nothing unusual is happening**, and the discomfort is unfamiliarity rather than a logical difficulty.

And the construction of D from H is mechanical. If H existed as a program, D would be a twenty-line wrapper around it — **no cleverness, no insight, and no appeal to anything exotic is required to build D.** Which is precisely the property the proof needs.

Which is why the contradiction must fall on H. D's existence follows from H's by construction; D cannot exist; therefore H cannot — and that logical shape is what the proof actually is, underneath the self-reference that draws all the attention.

2 The consequences

HALT IS RECOGNISABLE, AND ITS COMPLEMENT IS NOT

HALT is recognisable: simulate M on w using the universal machine (Module 04 §3). If the simulation halts, accept. **If M never halts, the recogniser runs forever** — which is exactly what a recogniser is permitted to do.

So HALT is recognisable and not decidable, which **proves that the two classes of Module 04 §2 genuinely differ** and is the first witness to that distinction.

And the complement of HALT is not even recognisable. The argument is one line: if it were, then HALT and its complement would both be recognisable, so by Module 04 §2's theorem HALT would be decidable — and §1 proved it is not.

So 'M does not halt on w' has no finite certificate. There is nothing that could be shown to you which would verify non-termination in general — no proof, no trace, no invariant, nothing of finite size that works for every non-terminating program. **Which is a substantially stronger and more unsettling statement than undecidability alone**, and it is why proving termination is harder than proving almost anything else.

Language	Status	Why
HALT	Recognisable, undecidable.	Simulate; the simulation may not terminate.

Language	Status	Why
The complement of HALT	Not recognisable.	Otherwise HALT would be decidable (the callout).
A_{TM} — does M accept w ?	Recognisable, undecidable.	The same proof, and this is the version Sipser uses.
E_{TM} — does M accept nothing at all?	Not recognisable.	A yes requires <i>every</i> input to fail, which is an infinite condition with no finite witness.
EQ_{TM} — do M_1 and M_2 accept the same language?	Neither it nor its complement is recognisable.	Strictly harder than halting, which Module 07's hierarchy makes precise — 'undecidable' is not one level but infinitely many.

3 Diagonalisation as a method

DIAGONALISATION: BUILD THE THING THAT DIFFERS FROM EVERYTHING IN THE LIST

Cantor: given a purported list of all real numbers, construct one that differs from the n th in the n th decimal digit. **It is not on the list, so no such list exists, so the reals are uncountable.**

Halting: given a purported decider, construct a machine that differs from whatever the decider predicts about it. **So no such decider exists.** §1 is Cantor's argument with programs in place of digits.

Gödel: given a formal proof system, construct a sentence asserting its own unprovability (Module 10) — the same construction, with provability in place of halting.

Russell's paradox, the time and space hierarchy theorems (CSCE 637), Rice's theorem (Module 06), and the undecidability of first-order validity are all the same method: assume an enumeration, and construct the element it misses. Learning it once is learning a large fraction of theoretical computer science, which is why §1 is worth reconstructing from memory rather than merely reading.

4 What the theorem forbids

- **No single program decides halting for all (program, input) pairs. That is the claim, exactly,** and nothing more follows from it than that.
- **It does not say that your program's termination is unknowable. Most programs are easy** — a loop with a decrementing bounded counter obviously terminates, and **a tool that handles the easy cases is genuinely valuable** (Module 12 §1).
- **It does not forbid a partial decider** that answers yes, no, or 'I cannot determine this'. **Every termination checker and every static analyser is one of these,** and the third answer is what buys decidability.
- **It does not forbid deciding halting for a restricted class of programs** — primitive recursive functions (Module 09 §1), total functional languages, programs whose loops have statically bounded counts. **All decidable, by construction,** which is Module 12 §2's other engineering response.
- **And it does not depend on the machine model** (Module 04 §1), **so no programming language, no hardware, and no future architecture escapes it** — including quantum (Module 11 §4). **'Undecidable' is about the universal quantifier,** and **every practical response in Module 12 works by weakening it:** fewer inputs, or a third answer.

WHY THIS IS THE MOST IMPORTANT RESULT IN THE PROGRAM

Everything else in this program could be superseded. A better algorithm, a larger model, a new architecture, a faster renderer — all of it is contingent on the state of the art.

This cannot be. It is a proof that no program exists, and it will be as true in a century as it is now — which is a kind of claim nothing else in twenty-two courses has made.

And it has immediate practical content. Module 06 turns it into undecidability for almost every interesting property of programs (Rice's theorem), **and Module 08 finds those properties in tools you use daily.**

So this is not philosophy. It is the reason your type checker rejects programs that would have worked, your optimiser misses optimisations that were valid, and your race detector reports races that cannot occur — and knowing that is the difference between being surprised by your tools and understanding them.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — chapter 4	The diagonalisation argument and the undecidability of A_{TM} , with the countability preliminaries of Module 04 §4.
MIT 18.404J — lectures 8–9 (free)	Sipser proving this himself , which is worth watching even if you have read the chapter.
Turing — On Computable Numbers, section 8 (free)	The original undecidability argument , framed in terms of circle-free machines rather than halting.
Hofstadter — Gödel, Escher, Bach	§3's method, explored at length and informally — not a textbook, and a good companion to Module 10.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write the halting proof out from memory** and check it against the text.
- 2 Identify exactly which step requires the encoding** of Module 04.
- 3 Explain to someone else why the contradiction blames H and not D**, and note where they object.
- 4 Implement a partial halting checker** that answers yes, no, or unknown, and run it on ten programs.

- 5 **Report how many it could decide**, and construct one it cannot.
- 6 **Write a recogniser for HALT** and demonstrate it running forever on a non-halting input.
- 7 **Prove the complement of HALT is not recognisable**, in your own words.
- 8 **Write out Cantor's diagonal argument** and map each step onto the halting proof.
- 9 **State the halting theorem precisely**, then write four things it does not say.
- 10 **Find a programming language in which all programs terminate** and explain what it gave up.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the halting proof in full.
- 2 Why does the contradiction fall on H?
- 3 Why is the self-reference not a trick?
- 4 Why is HALT recognisable?
- 5 Prove its complement is not recognisable.
- 6 What does it mean that non-termination has no finite certificate?
- 7 Place five languages in the landscape table.
- 8 State the diagonalisation method and name four theorems that use it.
- 9 Give five things the halting theorem does not forbid.